GEORG-AUGUST-UNIVERSITÄT
GÖTTINGEN

ISSN 1612-6793

ROBOGUARD

LEON-KARIM PREUSSE

Praktikumsbericht

27.03.2026

Institute of Computer Science
Computer Security and Privacy

RoboGuard
Forschungsbezogenes Praktikum
CSP-2026

Submitted by Leon-Karim Preuße Date of submission: 27.03.2026

First examiner/Erstgutachter: Prof. Dr.-Ing. Delphine Reinhardt
Supervisor/Betreuer: Sebastian Reinke

Georg-August-Universität Göttingen
Institute of Computer Science
Computer Security and Privacy

ZUSAMMENFASSUNG

Der Einsatz von sozialen Robotern löst bei vielen Menschen erhebliche Privatheitsbedenken aus, trotz ihres Potentials zur Unterstützung in verschiedenen Lebensbereichen. Bestehende Lösungen zur Kontrolle des Roboters und der eigenen Privatheit sind oftmals auf einzelne Anwendungen reduziert und nutzen zentrale Server. Im Rahmen dieser Arbeit wurde darum die Anwendung RoboGuard entwickelt, die sowohl auf einem android-basierten Roboter, als auch auf einem android-basierten Smartphone installiert werden kann. RoboGuard nutzt einen dezentralen, sicherheitsorientierten Ansatz zum Übertragen von anpassbaren Privatheitseinstellungen auf den Roboter. Technisch wird dies durch eine API und einen Server auf Roboterseite realisiert, über den eigens angepasste Steuerungsoptionen an das Smartphone übermittelt werden können. Auf Smartphone Seite wird die Anwendungs-Oberfläche mittels Jetpack Compose automatisch aus diesen Optionen generiert, über die sich Einstellungen vornehmen und zurück an den Roboter übertragen lassen. Zur Sicherstellung der Datenintegrität und Vertraulichkeit wurde ein dezentrales Kommunikationsmodell implementiert: Der Roboter fungiert als lokaler Server mit der Netty-Engine, während die Kopplung sicher über QR-Code-basiertes Certificate Pinning und One-Time-Passwörter (OTP) erfolgt. Die Authentifizierung der Datenpakete wird über HMAC-Verfahren gewährleistet, wobei sensible Schlüsselinformationen hardwarenah im Android Keystore gesichert werden. Die Arbeit zeigt sowohl, warum derartige Kontrollmöglichkeiten Privatheitsbedenken gegenüber sozialen Robotern mindern könnten, als auch welche Vorteile dezentrale und generalisierbare Infrastruktur bietet.

ERKLÄRUNG ZUM PRAKTIKUMSBERICHT

Hiermit versichere ich, die vorliegende Master-Thesis ohne Hilfe Dritter nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Alle Stellen, die aus Quellen entnommen wurden, sind als solche kenntlich gemacht. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

DECLARATION ON THE USE OF CHATGPT AND COMPARABLE TOOLS IN THE CONTEXT OF EXAMINATIONS

In this work I have used ChatGPT or another AI as follows:

- ☐ Not at all
- ☑ During brainstorming
- ☐ When creating the outline
- ☐ To write individual passages, altogether to the extent of 0% of the entire text
- ☑ For the development of software source texts
- ☑ For optimizing or restructuring software source texts
- ☑ For proofreading or optimizing
- ☑ Further, namely: The Documentation was done by AI. software source code was mostly written by hand, the exception being for example, but not only, repetitive tasks in the UI. Further, AI was used to generate Latex code for this document (only for formatting, no content).

I hereby declare that I have stated all uses completely.
Missing or incorrect information will be considered as an attempt to cheat.

Göttingen, 27.03.2026

Leon-Karim Preuße

INHALTSVERZEICHNIS

1 MOTIVATION 1
2 RELATED WORK UND STAND DER TECHNIK 3
3 WAHL DES SPEZIFISCHEN STACKS 4
4 METHODIK 5
4.1 System Design 5
4.1.1 Datenübertragung 5
4.1.2 Speicherkonzepte 7
4.1.3 Weitere Sicherheitsaspekte 7
4.1.4 Interface 8
5 IMPLEMENTIERUNG 9
5.1 Client 9
5.1.1 MainActivity 9
5.1.2 Das RobotAPI-Modul 9
5.2 Roboter/Server Seite 9
6 TESTS UND ERGEBNISSE 12
7 DISKUSSION & FUTURE WORK 13
8 FAZIT 14
8.1 Ergebniszusammenfassung 14
8.2 Ausblick und Weiterentwicklung 14

LITERATURVERZEICHNIS 15

1 MOTIVATION

Soziale Roboter sind, in Abgrenzung zu Servicerobotern, auf Interaktion und Kommunikation ausgelegt, nicht nur auf die reine Verrichtung einer konkreten Aufgabe. So könnten sie beispielsweise gegen die Folgen sozialer Isolierung von älteren Menschen eingesetzt werden [14],[6]. Ein solcher Einsatz dringt stark in die Privatsphäre von Nutzenden ein. Diese sozialen Roboter müssten sich im Zuhause der Menschen bewegen und dort verschiedenste Daten sammeln, mindestens Sensordaten, wie beispielsweise von Kamera und Mikrofon und diese irgendeiner Form verarbeiten. Es ist bereits aus verschiedenen Studien bekannt, dass der Einsatz von Robotern in einem so privaten Umfeld, wie dem eigenen Wohnort, auf Skepsis und Privatheitsbedenken stößt. Diese Bedenken variieren im Bezug auf Ort, Aktivität und Art der verarbeiteten Daten im eigenen Zuhause, scheinen aber im Allgemeinen sehr präsent zu sein. So gaben in einer Befragung 90% der Befragten an „extremely concerned“ zu sein, wenn der Roboter beispielsweise Gesicht und Stimme aufnimmt [1]. Ein Lösungsansatz für derartige Bedenken könnte sein, den Nutzenden möglichst viel Kontrolle über die vom Roboter erhobenen Daten, die Verarbeitungsform und die Reaktionen des Roboters zu ermöglichen. Darum wurde im Rahmen dieser Arbeit die Android–Anwendung RoboGuard entwickelt. Nutzer*innen der Anwendung können verschiedene Einstellungen vornehmen, die das Verhalten eines sozialen Roboters in Bezug auf die Privatsphäre der Nutzenden beeinflusst. Das soll die Kontrolle der Nutzer*innen über die Daten die der Roboter sammelt erhöhen – sei es in welchen Bereichen welche Sensoren verwendet werden dürfen oder ob bestimmte Technologien eingesetzt werden sollen, um das Verhalten des Roboters in bestimmten Situationen zu ändern. Um den Nutzer*innen die gewünschte Kontrolle zu ermöglichen, muss die Anwendung möglichst nutzerfreundlich gestaltet und leicht zu bedienen sein. Außerdem soll der Roboter Feedback auf geänderte Einstellungen geben. Smartphones eignen sich gut als Endgerät und stoßen laut Albuluwi auf hohe Akzeptanz für Anwendungen zur Kontrolle von Robotern. Daraus resultieren folgende Anforderungen:
Anforderungen:

- Nutzerfreundliche, Android-basierte App mit Interface für Privatheitseinstellungen
- Abdeckung diverser typischer Roboter-Daten
- Übertragung und Speicherung vom Smartphone auf den Roboter
- Visuelles Feedback durch den Roboter
- Kopplung der App (z. B. durch QR-Code)
- Übertragung durch öffentliches Netz (verschlüsselt)
- Kopplung neuer Geräte nur mit Passwort (nach Erst-Kopplung)
- Barrierefreiheit der Anwendung

- Kompatibilität mit nicht Android-basierten Systemen (z. B. iOS)
- Verbesserung der Nutzungsfreundlichkeit

Da die Anwendung die Verarbeitung sensibler Daten betrifft, müssen verschiedene Sicherheitsstandards erfüllt werden, vornehmlich muss die Integrität, Authentizität, Vertraulichkeit und Verfügbarkeit der Anwendung und der übertragenen Daten gewährleistet sein. Im Laufe des Entwicklungsprozesses, wurde sich daher für einen dezentralen Ansatz entschieden, der ohne zentralen Server auskommt und weder Roboter noch Smartphone des Nutzenden direkt über das Internet adressierbar macht. Dementsprechend ist die Anforderung „Übertragung durch öffentliches Netz (verschlüsselt)“ bereits zu Beginn der Entwicklung weggefallen. Auch wurde die Anwendung aus Zeitgründen nicht iOS kompatibel entwickelt und die Kopplung neuer Geräte erfolgt ohne Passwort um die Nutzung möglichst einfach zu gestalten. Die konkreten Entscheidungen unter anderem im Bezug auf die Anforderungen werden im weiteren Verlauf dieser Arbeit diskutiert werden. Außerdem ist ein Ziel der Anwendung möglichst allgemein nutzbar und nicht nur auf einen Roboter beschränkt zu sein. Eine API soll auf Roboterseite ermöglichen, dass andere Anwendungen je nach Bedarf festlegen können, welche Einstellungen möglich sind. Diese Optionen müssen an die Smartphone-Anwendung geschickt werden und das entsprechende Interface muss dynamisch generiert werden.

2 RELATED WORK UND STAND DER TECHNIK

Privatheitsbedenken und Sicherheit von Robotern ist bereits teilweise erforscht, es sind allerdings Lücken feststellbar. Aktuell forscht beispielsweise die Fachgruppe für Computersicherheit und Privatheit aktuell im Rahmen des Projekts PRIBONY zu den Privatheitsbedenken älterer Menschen gegenüber sozialen Robotern, da diese eher als Nebenaspekte bisheriger Studien erfasst wurden [9]. Gänzlich unerforscht sind diese Bedenken nicht. So zeigen sich Unterschiede in den Privatheitsbedenken bei der Anwesenheit von Robotern in verschiedenen Situationen (wie Zähne putzen, Essen, duschen) und Räumen, aber auch in Bezug auf verschiedene Aufgaben des Roboters (An Medikamente erinnern, Notruf wählen) [1]. Daraus ergibt sich die Notwendigkeit, dass der*die Nutzer*in das Verhalten des Roboters in Bezug auf verschiedene Situationen und Orte anpassen kann. Zudem wird in [1] ein System names PrivateLoc zur Steuerung eines Roboters skizziert, das durchaus Ähnlichkeiten zu RoboGuard aufweist und einige der Funktionalitäten von RoboGuard umfasst. Insbesondere können Steuersignale über eine Smartwatch an den Roboter gesendet werden, um ihn auf bestimmte Räume zu beschränken (Do-Not-Disturb-Mode). Vergleichbar zu anderen Architekturen, werden die Steuersignale erst von einem zentralen Server verarbeitet, bevor sie an den Roboter geschickt werden [1]. Das reduziert einerseits die benötigte Rechenleistung der Smartwatch und des Roboters, andererseits dient der Server als Proxy und verhindert dass über die IP Header der Pakete nachvollzogen werden kann, welche Smartwatch mit welchem Roboter kommuniziert. Privatheitseinstellungen wie der Do-Not-Disturb-Mode sind, sofern sie in Projekten realisiert werden, hoch anwendungsspezifisch, oft auf bestimmte Roboter ausgelegt und umfassen nur Teilaspekte von Privatheit. Direkte Kontrolle über die Sensoren, die der Roboter nutzen darf, bleibt beispielsweise aus, obwohl Albuluwis Erhebung durchaus ergibt, dass starke Bedenken bzgl. der Verarbeitung von Daten wie Bild und Ton durch den Roboter zu existieren scheinen. Implementiert jede Anwendung ihre eigene Möglichkeit Einstellungen vorzunehmen, sähen Nutzer*innen sich schnell mit einer Vielzahl an Kontrollanwendungen konfrontiert. RoboGuard versucht diese Lücke zu schließen, in dem es eine API bietet die Anwendungen nutzen können, um die Einstellungen die RoboGuard der Nutzer*in ermöglicht individuell zu bearbeiten. So kann RoboGuard als zentraler Hub dienen, je nach Roboter können beispielsweise unterschiedliche Sensoren eingestellt. oder für bestimmte Anwendungen bestimmte Verhaltensweisen des Roboters kontrolliert werden. Zudem skaliert ein zentraler Server schlecht mit einer hohen Anzahl an Nutzenden und stellt als Single-Point-of-Failure ein hohes Risiko für die Vertraulichkeit sämtlicher auf ihm gespeicherten Daten, sowie für die Verfügubarkeit des Dienstes dar. Dezentrale Lösungen könnten derartige Probleme vorbeugen, scheinen nach aktuellem Stand der Forschung allerdings kaum berücksichtigt zu sein, genauso wenig wie generalisierbare Systeme. Viel Forschung zu Privatheitsaspekten von sozialen Robotern konzentriert sich außerdem auf Deep Learning, die automatisierte Erkennung als besonders privater verstandener Situationen oder allgemeine Ansätze in Bezug auf sichere Implementierung von Software für Roboter [2, 19, 14, 13, 10, 17, 16].

3 WAHL DES SPEZIFISCHEN STACKS

Android 9 wurde als Betriebssystem gewählt. Es läuft auf den meisten Robotern, da es quelloffen ist und auch auf den meisten mobilen Systemen. Als Sprache wurde Kotlin verwendet. Kotlin macht den Einstieg in die Android Entwicklung durch intuitive Syntax und gute Dokumentation leicht, bietet für RoboGuard geeignete Pakete und ist performant genug für die Anwendung [5]. Zwecks Datenübertragung läuft auf dem Roboter ein Server basierend auf KTOR und der Netty-Engine. Netty läuft sehr stabil auf dieser Android Version, außerdem hat sich in der Entwicklung gezeigt, das anderen Engines teilweise Binaries auf dem Roboter fehlen. Netty unterstützt zudem HTTPS zur Verschlüsselung und Datenübertragung [12]. Die Verwendung von HTTPS vereinfacht die verschlüsselte Datenübertragung, da HTTPS als sicher und gut getestet gilt. Um HTTP-Anfragen an den Server zu stellen, wurde OKHTTP verwendet, da OKHTTPS ermöglicht einen eigenen Trust Manager zur Zertifikatsverwaltung zu implementieren [11]. Das Interface wurde mit Jetpack Compose geschrieben. Jetpack Compose ist ein modernes von Google entwickeltes Framework für Interfaces. Jetpack Compose ermöglicht einen guten Einstieg im Vergleich zu veralteten XML Interfaces und leichteres Troubleshooting in einem modernen, state-of-the-art Framework [4].

4 METHODIK

4.1 SYSTEM DESIGN

Entsprechend der Anforderungen, ist der wichtigste Aspekt von RoboGuard die Datenübertragung. Um die Datenübertragung zu realisieren, musste RoboGuard in zwei Applikationen aufgeteilt werden. Eine der Applikationen läuft auf dem Roboter, eine auf dem Smartphone der Nutzer*in (im Folgenden als „Client“). Das System musste dementsprechend bezüglich folgender Aspekte entworfen werden:

4.1.1 *Datenübertragung*

Die Datenübertragung ist das Kernelement von RoboGuard. Um eine Übertragung zu gewährleisten, die möglichst unabhängig von räumlicher Distanz ist, wird eine Internetverbindung genutzt, anstelle von alternativen Optionen wie Bluetooth. Um einen zentralen Server als Single-Point-of-Failure zu vermeiden, wurde sich für einen dezentralen Ansatz entschieden, in dem der Roboter einen eigenen Server hostet. Überlastung, Datenverlust oder Verlust der Verfügbarkeit eines zentralen Servers betrifft alle Nutzenden, tritt eines dieser Szenarien im dezentralen Szenario ein, betrifft es nur einzelne Nutzende. Der Roboter ist außerdem nur über das lokale Netzwerk erreichbar, mit dem er verbunden ist. Da der Roboter durch den Router im lokalen Netz eine lokale IP erhält, ist er nur im lokalen Netz über diese adressierbar, der Router dient als Proxy und schützt so den Roboter vor externem Zugriff [15]. Zudem befinden sich aller Wahrscheinlichkeit nach sowohl Nutzer*innen als auch Roboter ohnehin im selben lokalen Netz, wenn Bedarf zur Steuerung bzw. Änderung an Privatheitseinstellungen des Roboters entsteht. Der Server auf dem Roboter (im Folgenden nur „Server“) ist ausschließlich über das HTTPS-Protokoll erreichbar. HTTP ist der Standard für Datentransfer über Netzwerke und HTTPS bietet zusätzliche Verschlüsselungs-, Authentifizierungs- und Integritätsmechanismen. Die eigene Entwicklung solcher Sicherheitsmechanismen und insbesondere das Testen von ihnen ist komplex. HTTPS ist erwiesenermaßen zuverlässig und eine bereits erprobte Lösung [15]. Die Daten werden als JSON String codiert übertragen, um eine einfache Verarbeitung zu gewährleisten. HTTPS basiert auf Requests und Responses, d.h. der Client sendet eine Anfrage an den Server und bekommt dafür eine Antwort. Um dieses Anfragen stellen zu können, benötigt der Client jedoch

1. IP und Port unter denen der Server im Netzwerk erreichbar ist
2. . das Zertifikat des Servers um verschlüsselt kommunizieren zu können.

Sobald der Server startet, wird entweder ein selbst-signiertes Zertifikat erstellt oder ein bereits erstelltes Zertifikat geladen. Der Server ist nach Start nur mit seinem Zertifikat und seiner IP erreichbar. Da die meisten modernen Netzwerke das Dynamic Host Configuration Protocol (DHCP) nutzen, reicht es nicht, dem Client einfach IP und Zertifikat mitzuteilen. DHCP weist Geräten im Netzwerk dynamisch eine IP zu. Das bedeutet, dass der Roboter unter Umständen unter unterschiedlichen IPs erreichbar ist.

Dieses Problem wird durch Multicast-DNS gelöst. Der Server weißt sich selbst eine DNS-Adresse zu und flutet das Netzwerk mit der Information, welche IP sich hinter dieser DNS-Adresse verbirgt. Der Client löst vor jeder Anfrage die DNS-Adresse in die aktuelle IP auf [15]. Sobald der Server gestartet ist, codiert er die notwendigen Informationen (DNS-Adresse, Zertifikat und ein One-Time-Password [OTP]) als QR Code. Der Client kann die Informationen aus diesem QR-Code extrahieren und nutzen um die erste Verbindung aufzubauen. Da die OTPs ablaufen, können sie nicht für jede Verbindung genutzt werden um den Client gegenüber dem Server zu authentifizieren. Darum besteht der erste Verbindungsversuch aus einem Handshake. Der Client schickt sein OTP an den Server. Kann dieser das OTP verifizieren, antwortet er mit einem serverseitig generierten Shared Secret und einer ID, die der Server dem Client zuweist. Für jeden weiteren Verbindungsversuch, in dem tatsächlich Daten übertragen werden, hängt der Client seine ID und einen Hash aus Shared Secret und der gesendeten Nachricht an die eigentliche Nachricht an. Der Server kann diesen Hash ebenfalls erstellen und so den Client authentifizieren. Vor jeder Datenübertragung und am Ende vom Handshake, fragt der Client vom Server an, welche Daten (also welche Einstellungen) der Server empfangen kann und speichert die erhaltenen Informationen verschlüsselt ab.

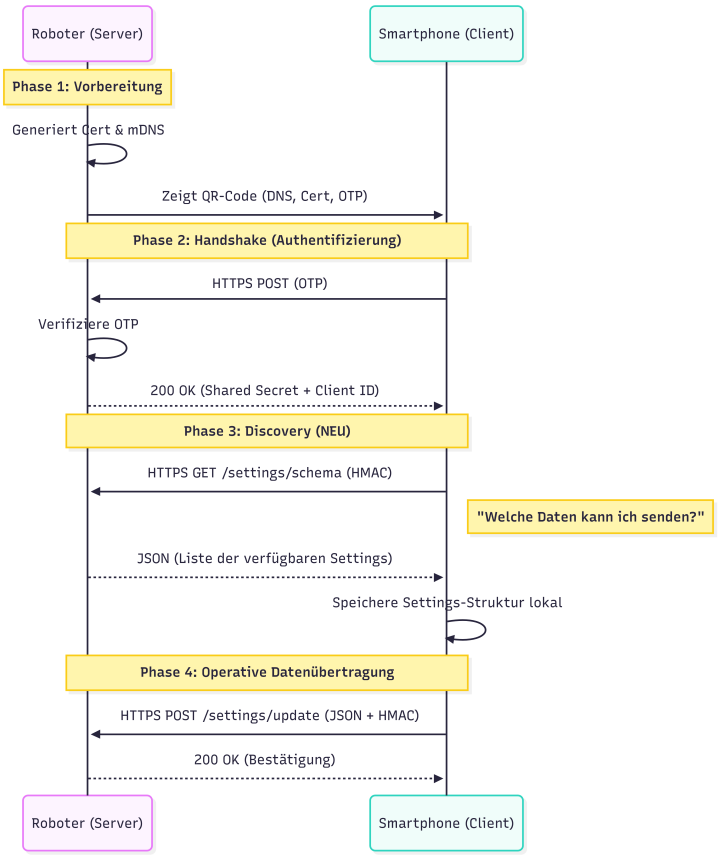

Abbildung 4.1: Netzwerkarchitektur von RoboGuard

4.1.2 *Speicherkonzepte*

Der Server muss die übertragenen Daten abspeichern. Da es sich bei diesen Daten um Einstellungen handelt, die eventuell andere Applikationen betreffen, werden diese Daten unverschlüsselt abgespeichert um diesen Applikationen Zugriff zu erlauben. Andererseits muss er den privaten Schlüssel zu seinem Zertifikat, das Zertifikat, das Shared Secret + die Client ID authentifizierter Clients, sowie die Server DNS Adresse und weitere persistente Variablen speichern. Bei all diesen Daten handelt es sich um vertrauliche Daten, auf die unberechtigter Zugriff möglichst verhindert werden sollte. Die Paare Client ID + Shared Secret werden deshalb in einer verschlüsselten SQL Datenbank gespeichert. Der Schlüssel für diese Datenbank liegt im Android Keystore. Der Android Keystore verhindert Zugriff anderer Applikationen auf die von ihm gespeicherten Schlüssel, selbst mit Root Rechten [7]. Darum wird das Zertifikat, sowie der zugehörige private Schlüssel und das zugehörige Passwort ebenfalls im Android Key Store gespeichert. Die DNS Adresse, sowie alle weiteren Variablen werden verschlüsselt in den EncryptedSharedPrefs gespeichert, die Schlüssel dafür liegen ebenfalls im Android Key Store. Auf Client Seite, wird ebenfalls das Shared Secret, sowie die Client ID verschlüsselt in den EncryptedSharedPrefs gespeichert, genau wie das Zertifikat, die DNS Adresse und die vom Roboter erhaltenen Einstellungen. An dieser Stelle sollte angemerkt werden, dass das Paket „EncryptedSharedPrefs“ deprecated ist und der Jetpack Data Store die zukunftssichere Wahl gewesen wäre. Allerdings ist das Problem der EncryptedSharedPrefs vor allem, dass sie den Main Thread blockieren, was für die Performanz von RoboGuard keine weitere Rolle spielt und die EncryptedSharedPrefs zu einer validen, wenn auch weniger modernen Wahl macht [3].

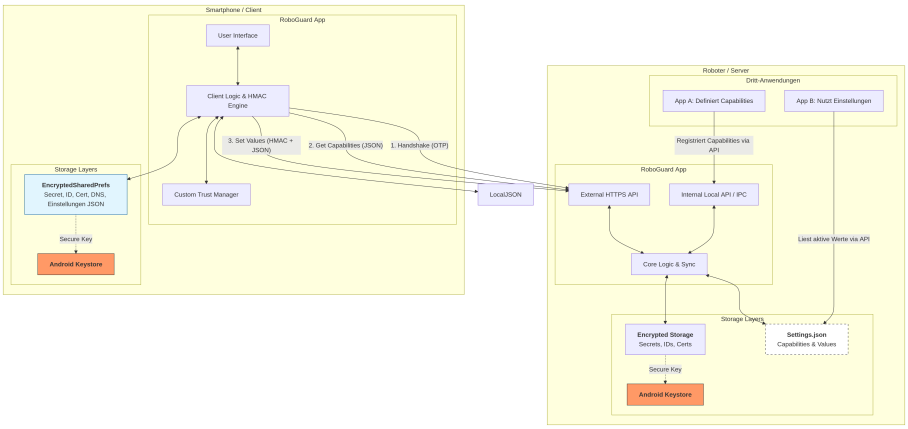

Abbildung 4.2: Datenspeicherung von RoboGuard

4.1.3 *Weitere Sicherheitsaspekte*

Anstatt das Zertifikat vom Server durch eine Certificate Authority zu verifizieren, nutzt der Client Certificate Pinning. Der Client vergleicht lediglich, ob das Zertifikat des Servers dem Zertifikat aus dem QR Code enstpricht [6]. Das OTP soll verhindern, dass ein potentieller Angreifer an die IP und das Zertifikat des Roboters gelangen kann, beispielsweise über einen alten QR-Code und sich daraufhin dauerhaft authentifizieren kann. Nur mit einem aktuellen OTP, was Zugriff auf einen aktuellen QR-Code und damit auf

den Roboter erfordert, ist eine Authentifizierung möglich. RoboGuard verwendet außerdem HMAC zur Authentifizierung des Clients, weil es gemeinhin als schlechte Praxis gilt, Passwörter bzw. Shared Secrets direkt zu teilen. Wird die Vertraulichkeit einer einzelnen Nachricht kompromittiert in der das Shared Secret steht, wäre die Authentizität folgender Nachrichten nicht mehr gegeben [15]. Android Key Store verhindert auch im entschlüsselten Zustand externen Zugriff auf Anwendungsschlüssel unabhängig von den Berechtigungen, beispielsweise einer kompromittierten Anwendung. Die EncryptedSharedPrefs speichern deswegen den Schlüssel zu Ihrer Entschlüsselung ausschließlich im Android KeyStore.

4.1.4 *Interface*

Das Interface läuft parallel zu allen vorigen Funktionalitäten auf einem seperaten Thread und wartet nach Bedarf auf das Backend (auf Client Seite beispielsweise nach dem Koppeln und wenn der Versuch gestartet wird Daten zu übertragen, auf Server Seite muss vor allem der QR-Code abhängig vom Backend aktualisiert werden). Es zeigt Ansichten basierend auf Nutzer*innen-eingaben oder Backend-Informationen an. Es ist außerdem verantwortlich für visuelles Feedback bei erfolgreicher oder fehlerhafter Datenübertragung oder Kopplung.

5 IMPLEMENTIERUNG

Die Implementierung von RoboGuard folgt einer strikten Trennung zwischen der Benutzeroberfläche (Frontend) und der Kommunikationslogik (Backend).

5.1 CLIENT

5.1.1 *MainActivity*

Das Modul `MainActivity` ist für die Steuerung der Benutzeroberfläche und den grundlegenden Logikfluss der Anwendung verantwortlich. Um eine flüssige Nutzungserfahrung zu gewährleisten, laufen rechenintensive Operationen oder Netzwerkzugriffe der `RobotAPI` grundsätzlich in Hintergrund-Threads (via Kotlin Coroutines), während die MainActivity auf dem Main-Thread verbleibt. Der Ablauf wird durch den Kopplungsstatus bestimmt: Nachdem initial geprüft wird, ob bereits Kopplungsdaten existieren, kann der*die Nutzer*in bei Bedarft einen QR-Scanner starten, der mit `Google ML Kit` implementiert wurde. Aus einem gescannten QR-Code werden die zur Kopplung benötigten Parameter extrahiert. Nach erfolgreicher Kopplung wechselt die Ansicht dynamisch und zeigt die verfügbaren Robotereinstellungen an. `MainActivity` reagiert auf Exceptions und Fehler der `RobotAPI` und gibt dementsprechendes visuelles Feedback für die Nutzer*in.

5.1.2 *Das RobotAPI-Modul*

Das `RobotAPI-Modul` bildet den Kern der clientseitigen Anwendung. Es beinhaltet die Logik für die Kommunikation mit dem Roboter, Verschlüsselung von Daten, sowie der Kopplung. Außerdem implementiert `RobotAPI` einen `NSDManager` (Network Service Discovery). Schlägt die Verbindung zum Server über die gespeicherte IP fehl, wird über den DNS Namen des Roboters die aktuelle IP gefunden.

NETZWERK-ENGINE UND PROTOKOLLE Für den Datenaustausch wird der Ktor-HTTP-Client mit der OkHttp-Engine eingesetzt. Diese Wahl ermöglicht eine effiziente Handhabung von asynchronen Requests und über das Content Negotiation Plugin können Datenstrukturen automatisch als JSON serialisiert bzw. deserialisiert. Der OKHTTP-Client nutzt einen Trust Manager um Certificate Pinning zu realisieren.

5.2 ROBOTER/SERVER SEITE

Das Kernstück der Roboterseitigen Anwendung besteht aus dem Modul `RobotServerService` (im Folgenden als `Service`). Es handelt sich dabei um einen tatsächlichen Service, der separat von der `MainActivity`und der UI gestartet werden kann und das automatisch bei Systemstart tut. Auf diese Art und Weise kann der Roboter Daten empfangen, unabhängig davon, ob die eigentliche Anwendung gestartet oder offen ist. Beim ersten

Anwendungsstart, generiert der Service automatisch eine eigene mDNS-Adresse, sofern er keine aus den SharedPrefences laden kann. Der NSD Manager erzeugt einen Eintrag mit dieser Adresse und ermöglicht es die mDNS Adresse im Netzwerk zur IP des Roboters aufzulösen. Außerdem generiert der Service automatisch ein Selbst-signiertes X.509-Zertifikat, sofern kein bereits erstelltes Zertifikat verfügbar ist. Dafür wird die Bibliothek `BouncyCastle` genutzt, um die Einschränkungen der Standard-Android-API zur Schlüsselerstellung zur Laufzeit zu umgehen. Diesem Zertifikat werden Subject Alternative Names (SANs) hinzugefügt, bestehend aus der eigentlichen IP, sowie der mDNS Adresse, da moderne Frameworks oft mindestens eine SAN als Sicherheitsstandard voraussetzen. Dieses Zertifikat wird verschlüsselt gespeichert, das Passwort liegt im AndroidKeyStore. Den Zugriff regelt das Modul Key-Manager. Der Server selbst basiert ebenfalls auf KTOR, nutzt allerdings die Netty Engine und wird ebenfalls nach dem Zertifikat und der mDNS-Adresse beim Servicestart geladen. Der Server ist unter fünf verschiedenen Endpunkten erreichbar:

Tabelle 5.1: Übersicht Server-Routing

Endpunkt	**Funktion**	**Sicherheitsstufe**
`/ping`	Erreichbarkeitsprüfung	HTTPS
`/update_capabilities`	Empfängt Einstellungs-Parameter lokaler Apps	Nur Localhost (127.0.0.1)
`/otp_auth`	Verarbeitet `secret_handshake()` via Auth-Modul. Liefert bei Erfolg Shared Secret.	OTP-Validierung, HTTPS
`/capabilities`	Liefert verfügbare Einstellungen an den Client	HMAC-Signatur, HTTPS
`/save`	Speichert empfangene Privatsphäre-Einstellungen als JSON	HMAC-Signatur, HTTPS

Das über den Endpunkt `/otp_auth` und das `Authentification`-Modul generierte Secret und die generierte Client ID werden in einer verschlüsselten SQL Datenbank über das Paket `Room` gespeichert. Die Steuerung findet im Modul `EncryptedDatabaseController` statt, das Passwort liegt im AndroidKeyStore. Der Server erzwingt HTTPS auf Port 8443 über das vorher generierte Zertifikat. Außerdem wurden für die letzten 3 Endpunkte eigene Routing-Erweiterungen implementiert (`secure_post()/secure_get()`). Diese prüfen bei jeder Request den Nachrichten-Header auf eine Client-ID, sowie einem gültigem Hash Wert aus Nachricht und Secret korrespondierend zur ID. Stimmen die Hashwerte nicht überein, wird die Verbindung abgebrochen. Der Server bringt das Interface in den Vordergrund und zeigt eine Benachrichtigung an, wenn Daten empfangen werden. Die Authentifizierungslogik wurde im `Authentification` Modul implementiert. Es für die Generierung und Validierung der OTPs und der Shared Secrets verantwortlich. Die

`MainActivity` entält das Interface mit dem QR Code, sowie eine zweite Ansicht, die die aktuellen Roboter-Einstellungen enthält.

6 TESTS UND ERGEBNISSE

Beide Anwendungen wurden manuell getestet, die Roboter-seitige Anwendung auf dem Robotermodell Orionstar Greeting Mini im Labor der Forschungsgruppe für Computersicherheit und Privatheit der Georg-August-Universität Göttingen, die Client-seitige Anwendung auf einem Google Pixel 7a mit GrapheneOS und auf einem Samsung Galaxy S20+, beide auf dem respektiven Patch-Stand vom 26. März 2026. GrapheneOS ist ein Android Betriebssystem [8]. Folgende Fehlerszenarien wurden getestet:

1. Unterschiedliche Netzwerkverbindungen Roboter/Smartphone oder VPN:
   → Verbindung wird abgelehnt, tritt der Fehler beim Koppeln auf, fordert das Smartphone zum erneuten Koppeln auf. Tritt der Fehler bei der Übertragung der Einstellungen auf, zeigt das Smartphone lediglich eine Fehlermeldung.

2. Unterschiedliche Zertifikate→Verbindung wird abgelehnt, Fehlermeldungen wie bei 1.

3. Fehlgeschlagene Authentifizierung→Verbindung wird abgelehnt, Fehlermeldungen wie bei 1.

4. Verbindungsabbrüche→Werden abgefangen, Fehlermeldungen wie bei 1.

5. Neue Übertragung ohne neues Koppeln nach zwei Wochen

Der erste Verbindungsversuch ist selten fehleranfällig. Das stellt an sich kein größeres Problem dar, da Koppeln etwas einmaliges ist und konsistent ab dem zweiten Versuch funktioniert. Eine mögliche Erklärung wäre, dass der RobotServerService Zeit zum Start braucht und der erste Verbindungsversuch womöglich teilweise zu schnell erfolgt. Ansonsten scheint die Anwendung fehlerfrei zu funktionieren. Nutzer*innen werden über Bedienungsfehler (wie unterschiedliche Netzwerke) informiert und Fehler werden abgefangen, anstatt Abstürze zu verursachen. Die Fehlermeldungen könnten allerdings ausdifferenziert werden. Die Zeit zwischen Koppeln und Bestätigung für ein erfolgreiches Koppeln beträgt unter einer bis zu mehreren Sekunden. Die Datenübertragung läuft schneller, meist unter einer Sekunde. Das hängt davon ab, ob die mDNS Adresse aufgelöst werden muss, mit einer statischen IP läuft die Übertragung wesentlich schneller.

7 DISKUSSION & FUTURE WORK

Es wäre an sich durchaus möglich, Abstand vom dezentralen Konzept der Anwendung zu nehmen und den Roboter auch aus dem öffentlichen Netz erreichbar zu machen. Das geht zwar mit Risiken einher und der erhöhte Nutzen ist fraglich, wie bereits im Kapitel System Design beschrieben. Vor allem die Nutzerfreundlichkeit würde in so einem Fall aber wohl erhöht werden, da unterschiedliche Netzwerke keinen Verbindungsfehler werfen würden. Tatsächlich könnten über die Anwendung Tailscale beide Konzepte miteinander kombiniert werden. Tailscale nutzt UDP Port Punching, um zwei Endgeräten zu ermöglichen sich gegenseitig direkt adressieren zu können. Dafür wird die Firewall des Routers umgangen, in dem beide Seiten gleichzeitig einen Port öffnen [18]. Darüber kann ein sogenanntes Tailnet erstellt werden, Geräte in selben Tailnet können sich an der Firewall eines Routers vorbei direkt adressieren. Das Tailnet könnte bei Kopplung definiert werden und lediglich Server und Client enthalten. Somit wäre der Roboter für unbekannte Geräte außerhalb des Tailnets weiterhin nicht adressierbar und die Sicherheitsvorteile würden erhalten bleiben. Tailscale verfügt über die Libtailscale SDK eine Möglichkeit diese Funktionalität direkt in die Anwendung zu integrieren. Da die Implementierung nicht trivial ist und den Rahmen dieses Projektes überschritten hätte, wurde in diesem Fall darauf verzichtet. Weitere Verbesserungsmöglichkeiten lassen sich vor allem in Bezug auf API für andere Anwendungen finden. Aktuell erlaubt die API einfach das Bearbeiten der JSON Datei, d.h. Anwendungen können die Einstellungen anderer Anwendungen überschreiben. Zudem wäre es vermutlich sinnvoll, die API für lokale Anwendungen aus dem Netty Server auszulagern. Die API dient in diesem Fall vor allem als Platzhalter, um einen Ansatzpunkt für weitere Entwicklung zu bieten und wurde kaum getestet. Die API konkret auszubauen, war allerdings kein Teil der Anforderungen an dieses Projekt und hätte ebenfalls den Umfang dieses Projektes überschritten. Zuletzt wurde darauf verzichtet diese Anwendung iOS kompatibel zu gestalten, ebenfalls aufgrund des hohen Aufwands. IOS Kompatibilität würde die Anzahl an potentiellen Nutzenden stark erhöhen. Zuletzt hätte sich die Nutzerfreundlichkeit durch Animationen erhöhen lassen. Beide Punkte standen in diesem Fall hinter der sicheren Implementierung der Kernfunktionalitäten zurück.

8 FAZIT

8.1 ERGEBNISZUSAMMENFASSUNG

Die im Anforderungskatalog definierten Muss-Kriterien wurden vollständig erfüllt. RoboGuard bietet eine nutzerfreundliche, Android-basierte Schnittstelle, die typische und darüber hinausgehende Privatheitseinstellungen abdeckt. Die Kopplung via QR-Code ermöglicht einen sicheren Zertifikats- und OTP-Austausch, während das visuelle Feedback auf dem Roboter die notwendige Transparenz über vorgenommene Änderungen schafft. Dass die optionalen Anforderungen nur teilweise umgesetzt wurden, ist einer bewussten Priorisierung zugunsten der Systemsicherheit geschuldet. Die Implementierung einer verschlüsselten SQLite-Datenbank mit Room, sowie der weiteren verschlüsselten Speicherung von Daten, der komplexen Authentifizierungslogik, und der Absicherung des Ktor-Servers gegen verschiedene Angriffsszenarien, erforderte einen signifikanten Entwicklungsaufwand. Ein Verzicht auf diese Module hätte zwar den Funktionsumfang vergrößert, wäre jedoch dem sensiblen Charakter der verarbeiteten Daten und dem aktuellen Forschungsstand nicht gerecht geworden.

8.2 AUSBLICK UND WEITERENTWICKLUNG

Ein wesentlicher Erkenntnisgewinn während der Entwicklung bestand darin, die große Relevanz der Erweiterbarkeit der Anwendung zu verstehen Die Transformation von einer statischen App hin zu einer zentralen API erlaubt es zukünftigen Anwendungen auf dem Roboter, die für sie spezifischen, durch die Nutzer*in konfigurierbaren Einstellungen über RoboGuard einstellen zu lassen. Für die weitere Entwicklung bieten sich drei primäre Ansätze an:

1. Ausbau globaler Konnektivität mit Tailscale
2. iOS Kompatibilität
3. Ausbau der API (Verbesserung der Modularität)

Zusammenfassend lässt sich festhalten, dass RoboGuard eine architektonisch geeignete Basis die Kontrolle der Privatsphäre von Nutzenden in Bezug auf android-basierte Roboter darstellt. Die API ermöglicht einen modularen Ansatz, der es RoboGuard ohne weitere Änderungen erlaubt mit einem System mitzuwachsen.

LITERATURVERZEICHNIS

[1] Tahani Ali Albuluwi. Iot-based solutions for privacy-preservation during interaction with assistive robotics, May 2020. URL https://repository.fit.edu/etd/736.

[2] Y. Chen, Y. Ping, Z. Zhang, B. Gaucher, H. Lu, and Y. Chen. Privacy-preserving image multi-classification deep learning model in robot system of industrial IoT. *Neural Computing and Applications*, 33(10):4677–4694, 5 2021. doi: 10.1007/s00521-020-05426-0. URL https://doi.org/10.1007/s00521-020-05426-0.

[3] Android Developers. Encryptedsharedpreferences | android jetpack, 2024. URL https://developer.android.com/reference/androidx/security/crypto/EncryptedSharedPreferences. Teil der AndroidX Security-Bibliothek zur verschlüsselten Datenspeicherung.

[4] Google Developers. Thinking in jetpack compose, 2024. URL https://developer.android.com/develop/ui/compose/mental-model. Erklärt das zustandsbasierte UI-Modell.

[5] Google Developers. Android-first development with kotlin, 2024. URL https://developer.android.com/kotlin/first.

[6] D. Díaz-Sánchez, A. Marín-Lopez, F. A. Mendoza, P. A. Cabarcos, and R. S. Sherratt. TLS/PKI challenges and certificate pinning techniques for IoT and M2M secure communications. *IEEE Communications Surveys & Tutorials*, 21(4):3502–3531, October 2019. doi: 10.1109/COMST.2019.2914453.

[7] Google Developers. Android keystore system, 2024. URL https://developer.android.com/privacy-and-security/keystore. Zugriff am: 26.03.2024.

[8] GrapheneOS Project. Features overview, 2024. URL https://grapheneos.org/features. Detaillierte Übersicht der Sicherheits- und Datenschutzfunktionen von GrapheneOS.

[9] Georg-August-Universität Göttingen. Pribony: Verständnis der privatheitsbedenken älterer menschen in bezug auf roboter und erforschung von transparenz als lösung, 2025. URL https://www.uni-goettingen.de/de/687594.html. Gefördert durch die Deutsche Forschungsgemeinschaft (DFG) - Projektnummer 539453929.

[10] Tanja Heuer, Ina Schiering, and Reinhard Gerndt. Privacy-centered design for social robots. *Interaction Studies*, 20(3):509–529, November 2019. doi: 10.1075/is.18063.heu. URL https://doi.org/10.1075/is.18063.heu.

[11] Square Inc. Okhttp: An http client for android and java applications, 2024. URL https://square.github.io/okhttp/. Dokumentation der Client-Implementierung und TLS-Konfiguration.

[12] JetBrains. Ktor http server engines, 2024. URL https://ktor.io/docs/server-engines.html. Belegt die Unterstützung der Netty-Engine.

[13] Seyed Mohammad Sadegh Khaksar, Fatemeh S. Shahmehr, Shah Miah, Tugrul Daim, and Dilek Ozdemir. Privacy concerns versus personalisation benefits in social robot acceptance by employees: A paradox theory — contingency perspective. *Technological Forecasting and Social Change*, 198:123034, 2024. ISSN 0040-1625. doi: https://doi.org/10.1016/j.techfore.2023.123034. URL https://www.sciencedirect.com/science/article/pii/S0040162523007199.

[14] Oliver Korn. Soziale roboter – einführung und potenziale für pflege und gesundheit. *Wirtschaftsinformatik & Management*, 11(3):126–135, 3 2019. doi: 10.1365/s35764-019-00187-5. URL https://doi.org/10.1365/s35764-019-00187-5.

[15] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Pearson, Harlow, England, 8 edition, 2021. ISBN 9781292405513. URL https://elibrary.pearson.de/book/99.150005/9781292405513.

[16] Christoph Lutz and Aurelia Tamò-Enjoy. Privacy and social robots: A systematic review. *Information, Communication & Society*, 22(8):1087–1110, 2019. doi: 10.1080/1369118X.2017.1410202.

[17] Samson Ogheneovo Oruma, Yonas Zewdu Ayele, Fabien Sechi, and Hanne Rødsethol. Security aspects of social robots in public spaces: A systematic mapping study. *Sensors*, 23(19):8056, September 2023. doi: 10.3390/s23198056. URL https://doi.org/10.3390/s23198056.

[18] Tailscale. How NAT traversal works, April 2020. URL https://tailscale.com/blog/how-nat-traversal-works. Ein technischer Leitfaden zu UDP, STUN und ICE.

[19] B. Tang, D. Sullivan, B. Cagiltay, V. Chandrasekaran, K. Fawaz, and B. Mutlu. Confidant: A privacy controller for social robots. In *2022 17th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 205–214, Sapporo, Japan, March 2022. doi: 10.1109/HRI53351.2022.9889540.